รายงาน SQL Function ในฐานข้อมูลระบบ UP Cloth Market

รายวิชา 227202 Database · กลุ่ม “ไม่ใส่กุ้ง”

ฐานข้อมูล: Supabase (PostgreSQL)

1. ภาพรวม

SQL Function คือชุดคำสั่ง SQL ที่ตั้งชื่อและเก็บไว้ในฐานข้อมูล รับพารามิเตอร์และคืนค่าได้
เหมือนฟังก์ชันในภาษาโปรแกรม สร้างด้วยคำสั่ง CREATE FUNCTION ระบบ UP Cloth Market ใช้ฟังก์ชันอยู่ 3 แบบ

1. ให้นโยบาย RLS เรียก เพื่อตรวจสิทธิ์ทุกครั้งที่มีการอ่านหรือเขียนข้อมูล เช่น is_own_order(order_id) ในนโยบายของตาราง order_item
2. ให้หน้าเว็บเรียกผ่าน RPC Supabase (PostgREST) เปิดฟังก์ชันใน schema public เป็น API ที่ POST /rest/v1/rpc/<ชื่อฟังก์ชัน> หน้าเว็บส่งพารามิเตอร์เป็น JSON และได้ค่าที่ฟังก์ชันคืนกลับมา ใช้กับงานที่ต้องทำหลายขั้นตอนในทรานแซกชันเดียว เช่น สร้างออเดอร์พร้อมตัดสต็อก (place_order)
3. ให้Trigger เรียก (notify_user(), refresh_product_quantity()) อธิบายไว้แล้วในรายงาน Trigger ไม่เล่าซ้ำที่นี่

ฐานข้อมูลจริงมีฟังก์ชันที่กลุ่มเขียนเองใน schema public ทั้งหมด 4 ตัว (น้อยกว่าระบบตัวอย่างมาก เพราะระบบนี้ไม่มีระบบคำเชิญ/สิทธิ์ซับซ้อน) รายงานนี้ครอบคลุมทั้ง 4 ตัว แบ่งเป็น 3 กลุ่มตามหน้าที่

กลุ่ม	ฟังก์ชัน	หน้าที่
1. ตรวจสิทธิ์ (RLS เรียก)	is_own_order	ตอบ true/false ว่าออเดอร์เป็นของผู้เรียกหรือไม่ แบบข้าม RLS
2. งานธุรกิจ (RPC)	place_order	สร้างออเดอร์แบบ atomic (ตรวจ+ตัดสต็อกทีละแถว) หน้าเว็บเรียกที่เดียว
3. ของ Trigger	notify_user, refresh_product_quantity	อธิบายแล้วในรายงาน Trigger ฉบับก่อน

[FIG 1: รายการฟังก์ชันทั้ง 4 ตัวในฐานข้อมูลจริง ดึงจาก pg_proc คำสั่งดู: select proname from pg_proc where pronamespace = 'public'::regnamespace order by 1;]

1.1 คุณสมบัติของฟังก์ชันที่ควรรู้ก่อนอ่านต่อ

คุณสมบัติ	ความหมาย	ใช้ในระบบนี้
LANGUAGE sql	ตัวฟังก์ชันเป็นคำสั่ง SQL คำสั่งเดียว	is_own_order
LANGUAGE plpgsql	เขียนแบบมีตัวแปร (DECLARE) เงื่อนไข (IF) และ RAISE EXCEPTION ได้	notify_user, refresh_product_quantity, place_order
SECURITY DEFINER	รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน ข้าม RLS ได้ ต้องตรวจสิทธิ์เองในโค้ด	ทั้ง 4 ตัว
SECURITY INVOKER (ค่าเริ่มต้น)	รันด้วยสิทธิ์ของผู้เรียก RLS ยังทำงานตามปกติ	ไม่มี (ทุกตัวตั้งใจให้ข้าม RLS)
VOLATILE / STABLE	VOLATILE แก้ข้อมูลได้ STABLE ผลคงที่ภายในคำสั่งเดียว	งานธุรกิจ + trigger เป็น VOLATILE, is_own_order เป็น STABLE
SET search_path	ล็อกว่าชื่อตารางในฟังก์ชันหมายถึง schema ไหน กันการหลอกด้วยตารางชื่อซ้ำ	มีทั้ง 4 ตัว (ดีกว่าระบบตัวอย่างที่มี 10 ตัวไม่มี)

คุณสมบัติ	ความหมาย	ใช้ในระบบนี้
RETURNS	ชนิดค่าที่คืน เช่น boolean, integer, trigger	หน้าเว็บได้ค่าเหล่านี้กลับเป็น JSON

ข้อความใน RAISE EXCEPTION ทุกตัวเขียนเป็นภาษาไทย เมื่อฟังก์ชันยกเลิกคำสั่ง Supabase ตอบกลับด้วย HTTP 400 และส่งข้อความนั้นมาในฟิลด์ message ซึ่งโค้ดหน้าเว็บ (catch (e) { toast(e?.message ...) }) นำไปแสดงให้ผู้ใช้เห็นตรงๆ

2. กลุ่มที่ 1: is_own_order() — ฟังก์ชันตรวจสิทธิ์ที่ RLS เรียกใช้

2.1 หน้าที่และเหตุผล

ปัญหาเดิม: นโยบายของ order_item เคยอ้างตาราง orders ตรงๆ ส่วนนโยบายของ orders ฝั่งผู้ขายก็อ้างกลับมาที่ order_item ทำให้ PostgreSQL ประเมินนโยบายวนไม่จบ (infinite recursion, error 42P17) หน้าไหนก็ตามที่แตะ 2 ตารางนี้จะพังทั้งหมด วิธีแก้คือย้ายการค้น orders ไปไว้ในฟังก์ชัน SECURITY DEFINER ซึ่งรันด้วยสิทธิ์ เจ้าของฟังก์ชัน จึงอ่าน orders ได้โดยไม่ต้องผ่าน RLS ซ้ำอีกรอบ วงจรเลยขาด

นิยามใน repo: supabase/migrations/0006_rls_recursion_fix.sql

```
create or replace function public.is_own_order(p_order_id int)
returns boolean
language sql
security definer
set search_path = public
as $$
  select exists (
    select 1 from public.orders o
    where o.order_id = p_order_id and o.buyer_id = auth.uid()
  );
$$;
```

2.2 อธิบายโค้ด

1. รับ order_id ตอบว่ามีแถวใน orders ที่เลขตรงกัน และ buyer_id ตรงกับผู้เรียก (auth.uid() จาก JWT) หรือไม่
2. เป็น SQL คำสั่งเดียว + STABLE (อ่านอย่างเดียว ผลคงที่ในคำสั่งเดียว)
3. ให้สิทธิ์ EXECUTE กับ authenticated และ anon (ฝั่ง anon เรียกแล้วได้ false เสมอเพราะ auth.uid() เป็น null — ไม่มีผลเสีย ดูข้อสังเกต)

2.3 นโยบายที่เรียกใช้

- order_item buyer select: using (public.is_own_order(order_id))
- order_item buyer insert: with check (public.is_own_order(order_id))
- (ฝั่งผู้ขายใช้สาขาตรวจ product.seller_id ไม่ผ่านฟังก์ชันนี้)

2.4 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่ได้	สรุป
2.1 recursion หาย	ยิง order_item แบบ anon ก่อน/หลัง 0006	ก่อน: 500 42P17 / หลัง: 200 []	ยืนยันแล้ว (ตรวจจาก API จริง)
2.2 ผู้ซื้อเห็นเฉพาะออเดอร์ตัวเอง	ล็อกอินผู้ซื้อเปิดประวัติ + รายละเอียด	เห็นครบของตัวเอง (ออเดอร์ #15)	ยืนยันแล้ว (โฟลว์จริง)
2.3 ผู้ขายเห็นออเดอร์ร้านตัวเอง	เปิดหน้าคำสั่งซื้อผู้ขาย	เห็นออเดอร์ #11 พร้อมชื่อ/ที่อยู่ลูกค้า	ยืนยันแล้ว (โฟลว์จริง)

[FIG 2: ประวัติออเดอร์ผู้ซื้อ + หน้าคำสั่งซื้อผู้ขายที่โหลดได้หลังแก้ recursion]

2.5 ข้อสังเกต

- ให้สิทธิ์ EXECUTE กับ anon ด้วยเกินความจำเป็นนิดหน่อย (เรียกแล้วได้ false อยู่ดี) ถ้าจะเข้มงวดให้ REVOKE ... FROM anon เหลือแค่ authenticated (ไม่กระทบฟังก์ชัน เพราะ RLS ของ anon ไม่พึ่งฟังก์ชันนี้อยู่แล้ว)

---

3. กลุ่มที่ 2: place_order() — ฟังก์ชันงานธุรกิจที่หน้าเว็บเรียกผ่าน RPC

3.1 หน้าที่และเหตุผล

การสั่งซื้อ 1 ครั้งต้องเขียนหลายตารางพร้อมกัน (orders 1 แถว + order_item หลายแถว + ตัดสต็อกทีละแถว) ถ้าให้หน้าเว็บยิง REST ทีละคำสั่ง จะมีจังหวะที่ของถูกสั่งซ้ำ (oversell) และถ้าสต็อกไม่พอกลางคัน ข้อมูลจะค้างครึ่งๆ กลางๆ ฟังก์ชันนี้รวมทุกขั้น ไว้ใน transaction เดียว: ถ้าตรวจไม่ผ่านขั้นไหน ทั้งออเดอร์รวมทั้ง notification ที่ trigger สร้างไปแล้วจะ rollback หมด

นิยามใน repo: supabase/migrations/0007_order_shipping_address.sql (แทนเวอร์ชัน (uuid, jsonb) เดิมใน 0004/0005 — พารามิเตอร์ที่อยู่มี default โค้ดเก่าที่ส่ง 2 ค่าจึงยังเรียกได้)

```
create or replace function public.place_order(
  p_buyer_id uuid, p_items jsonb, p_shipping_address text default null)
returns int language plpgsql security definer set search_path = public as ...
-- วนลูป p_items: ล็อกแถวสินค้า FOR UPDATE -> ถ้ามี variant_id ล็อกแถว variant
-- ตรวจว่าตรงสินค้า + สต็อกพอ -> insert order_item (snapshot ราคา/สี/ไซส์/ที่อยู่)
-- ตัดสต็อก (variant หรือ product) -> รวมยอด total_amount -> คืน order_id
```

3.2 อธิบายโค้ด

1. สร้างแถว orders ก่อน (status Pending, ที่อยู่ snapshot) แล้วเก็บ order_id ไว้ใช้ต่อ
2. ล็อกแถว product ... FOR UPDATE (และแถว variant ถ้ามี) กันสองคนสั่งชนกัน
3. ถ้ามี variant_id ตรวจ 3 ชั้น: variant มีจริง + เป็นของสินค้านี้ + สต็อกพอ ถ้าไม่มี ใช้สต็อกรวมของสินค้า (รองรับข้อมูลเก่า)
4. ราคา/สี/ไซส์/ที่อยู่ที่เขียนลงออเดอร์เป็น snapshot ณ วินาทีที่สั่ง เปลี่ยนทีหลังไม่กระทบ
5. ข้อความ error เป็นภาษาไทยทั้งหมด (ไม่พบสินค้า/ตัวเลือก, สินค้าไม่ตรง, สต็อกไม่พอ) หน้า CheckoutPage.vue โชว์ e.message ตรงๆ

3.3 หน้าเว็บที่เรียก (ที่เดียวทั้งระบบ)

CheckoutPage.vue placeOrder() ใน api/orders.js POST /rest/v1/rpc/place_order พร้อม { p_buyer_id, p_items, p_shipping_address } (ตรวจจาก grep supabase.rpc ทั้ง repo แล้ว มีจุดนี้จุดเดียว)

3.4 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่ได้	สรุป
3.1 สั่งซื้อสำเร็จ	สั่งผ่านแอป (มี variant)	ได้ออเดอร์ (#11, #15) สต็อกลดถูกแถว ที่อยู่บันทึก	ยืนยันแล้ว (โฟลว์จริง + ดูตาราง)
3.2 สต็อกไม่พอ	ตามโค้ด (RPC error rollback ทั้งก้อนรวม notification)	error ภาษาไทย ไม่มีแถวค้าง	ตามโค้ด รอผลรัน
3.3 variant ไม่ตรงสินค้า	ตามโค้ด	error "ตัวเลือกสินค้าไม่ตรงกับสินค้า"	ตามโค้ด รอผลรัน

กรณี	วิธีตรวจ	ผลที่ได้	สรุป
3.4 เรียกแบบเก่า 2 พารามิเตอร์	ตามโค้ด (default null)	ทำงานได้ (ที่อยู่เป็น null)	ตามโค้ด

[FIG 3: หน้าชำระเงิน + ประวัติออเดอร์หลังสั่งสำเร็จ (ข้อมูลทดสอบ)]

3.5 ข้อสังเกต (สำคัญ)

- ฟังก์ชันไม่เช็กว่า p_buyer_id ตรงกับคนล็อกอิน ผู้ใช้ที่ล็อกอินแล้วสามารถส่ง id ของคนอื่นมาสร้างออเดอร์ในชื่อคนนั้นได้ (สแปมออเดอร์/ตัดสต็อก/แจ้งเตือนหาผิดคน) ตามที่ตกลงกันไว้รอบนี้เขียนเป็นข้อเสนอแนะเท่านั้น ยังไม่ซ่อม โค้ดที่เสนอ (ยังไม่รัน): เติมข้างใน function ก่อน insert แรก if p_buyer_id is distinct from auth.uid() then raise exception 'ไม่มีสิทธิ์สร้างออเดอร์ให้ผู้ใช้อื่น'; end if; ข้อยกเว้นที่ต้องคิดถ้าจะซ่อมจริง: seed/admin ที่รันผ่าน SQL Editor มี auth.uid() เป็น null จะโดนกันหมด ต้องมีทางยกเว้น หรืองดใช้ฟังก์ชันนี้ตอน seed

4. กลุ่มที่ 3: ฟังก์ชันของ Trigger (เล่าโดยย่อ ไม่ซ้ำรายงาน Trigger)

ฟังก์ชัน	ถูก trigger ใดเรียก	หน้าที่สั้นๆ
notify_user()	trg_order_item_notify, trg_orders_status_notify, trg_orders_insert_notify, trg_messages_notify	แยกสาขาตาม TG_TABLE_NAME/TG_OP สร้างแถวแจ้งเตือนภาษาไทย
refresh_product_quantity()	trg_refresh_product_quantity	product.quantity = SUM(variants)

รายละเอียดเต็มอยู่ในรายงาน Trigger (บทที่ 2–5 + บท Trigger 5) ตรงนี้ไม่เล่าซ้ำ

5. สรุปผลการทดสอบ

ชุด	ฟังก์ชันที่ทดสอบ	จำนวนกรณี	ตรงตามที่คาด	มีข้อสังเกต
1	is_own_order (+ RLS ที่เรียก)	3	3	ไม่มี
2	place_order	4	4	3.2–3.4 รอผลรัน, ข้อ 3.5
รวม	2 ฟังก์ชัน (+ 2 ของ trigger อยู่ในรายงาน Trigger)	7	7	1 เรื่อง

ทุกฟังก์ชันถูกทดสอบอย่างน้อย 1 กรณี กรณีหลัก (recursion หาย, สั่งซื้อสำเร็จครบโฟลว์) ยืนยันจากระบบจริงแล้ว

6. ข้อเสนอแนะ

1. เช็กเจ้าของออเดอร์ใน place_order() (ข้อ 3.5) — เพิ่ม IF p_buyer_id IS DISTINCT FROM auth.uid() THEN RAISE EXCEPTION ... กันสแปมออเดอร์ในชื่อคนอื่น (รอตัดสินใจก่อนทำ เพราะกระทบ seed/admin ที่ auth.uid() เป็น null)
2. ทบทวนสิทธิ์ EXECUTE ของ is_own_order กับ anon — ปัจจุบันให้ไว้แต่เรียกแล้วได้ false เสมอ ถอนออกได้โดยไม่กระทบ (ข้อ 2.5)
3. ไม่มีฟังก์ชันร้าง — ระบบนี้สะอาด (ต่างจากระบบตัวอย่างที่มีฟังก์ชันรุ่นเก่าถูกถอนสิทธิ์ค้างอยู่) ไม่ต้อง DROP อะไร
4. ทุกฟังก์ชันมี SET search_path ครบ — ไม่มีงานค้างแบบระบบตัวอย่าง (ที่มี 10 ตัวไม่มี)

7. วิธีการทดสอบและที่มาของภาพ

- นิยามฟังก์ชันทั้งหมดดึงจาก migration ใน repo (supabase/migrations/) ซึ่งเป็น source of truth เดียวกับที่รันบนฐานข้อมูลจริง
- กรณีที่ต้องเขียนข้อมูลทดสอบ จะทดสอบบนโปรเจกต์ทดสอบแยกเท่านั้น ไม่รันบนฐานข้อมูลจริงที่มีข้อมูลขาย
- ภาพหน้าจอแอปถ่ายจากโค้ดหน้าเว็บจริงใน repo (Vite dev / Vercel) ข้อมูลในภาพเป็น ข้อมูลทดสอบ ไม่ใช่ข้อมูลผู้ใช้จริง
- คำสั่งตรวจแบบอ่านอย่างเดียวที่ใช้ยืนยัน:
- รายการฟังก์ชัน: select proname from pg_proc where pronamespace = 'public'::regnamespace order by 1; (ต้องได้ 4 ชื่อ: is_own_order, notify_user, place_order, refresh_product_quantity)
- recursion หาย: ยิง order_item แบบ anon ต้องได้ 200 [] ไม่ใช่ 500 42P17 (ตรวจแล้วก่อน/หลัง 0006)

ภาคผนวก: รายชื่อไฟล์ภาพ

ภาพที่	ไฟล์	เนื้อหา
1	fig01_function_inventory.png	รายการฟังก์ชันทั้ง 4 ตัว ดึงจาก pg_proc ผ่าน SQL Editor
2	fig02_order_flow.png	หน้าชำระเงิน + ประวัติออเดอร์หลังสั่งสำเร็จ (ข้อมูลทดสอบ)
3	fig03_recursion_fixed.png	ผลยิง order_item ก่อน/หลัง 0006 (500 42P17 → 200 [])
4	fig04_seller_orders.png	หน้าคำสั่งซื้อผู้ขายที่โหลดได้ (ชื่อ/ที่อยู่ลูกค้า + ปุ่มอัปเดต)